

DATA STRUCTURES

Sessional Exam Study Notes | Units 1 to 5 | Beginner-friendly and past-paper driven

These notes cover **only your sessional syllabus**: Unit 1 (Introduction and Algorithm Analysis), Unit 2 (Arrays), Unit 3 (Stack), Unit 4 (Queue) and Unit 5 (Linked Lists). Every topic carries a priority tag taken from the past-paper analysis of the Jan-2022, Jan-2023, Jan-2025 and Dec-2025 papers.

Priority tag key

Tag	What it means
 VERY HIGH	Asked in all 4 papers. Master this first and practise by hand.
 HIGH	Asked in 2 or 3 of the 4 papers. Learn the full answer format.
 ESSENTIAL	Short or one-off questions that are easy marks, or the longest algorithm questions.
 SUPPORTING	Background theory. Read it so the tagged topics make sense.
 LOW	In the syllabus but never asked in 4 papers. Revise last.

Study roadmap (follow this order)

#	Topic	Tag	Why	Done
1	Infix to postfix and postfix evaluation (3.3, 3.4)	 VERY HIGH	Every paper, 6 to 8 marks	☐
2	Array address calculation (2.3)	 VERY HIGH	Same 6-mark numerical every paper	☐
3	Singly linked list algorithms and polynomial (2.4, 5.3)	 HIGH	Linked list is in all 4 papers	☐
4	Queue: simple queue, circular trace, static design (4.1 to 4.3)	 HIGH	Queue is in all 4 papers	☐
5	Unit 1 theory: linear vs non-linear, best/average/worst case (1.3, 1.7)	 HIGH	Easy 4 to 6 marks	☐
6	Circular list, doubly list, priority queue (5.4, 5.5, 4.4)	 ESSENTIAL	Longest algorithm questions (10 marks in Jan-23)	☐
7	Complexity summary table (6.1)	 ESSENTIAL	Cheap revision, 4 marks	☐
8	Sparse matrix, PUSH/POP/PEEP, prefix, recursion, RECOGNIZE (2.5, 3.1, 3.5 to 3.7)	 LOW	Never asked so far	☐

TIP

For every topic: (1) read the analogy, (2) follow the worked example with a pen, (3) close the notes and redo it, (4) try the practice question. Numericals and traces are learned by doing, not by reading.

Note on diagrams: ASCII figures sit in grey boxes in the Consolas font. If you edit this file, keep that font so the alignment holds.

UNIT 1: Introduction to Data Structures and Algorithm Analysis

1.1 What is Data and a Data Structure? SUPPORTING

Data is raw facts, such as numbers (45, 3.14) and text ("Riya"). A **data structure** is a way of organizing data in memory so that we can store it and use it efficiently.

ANALOGY

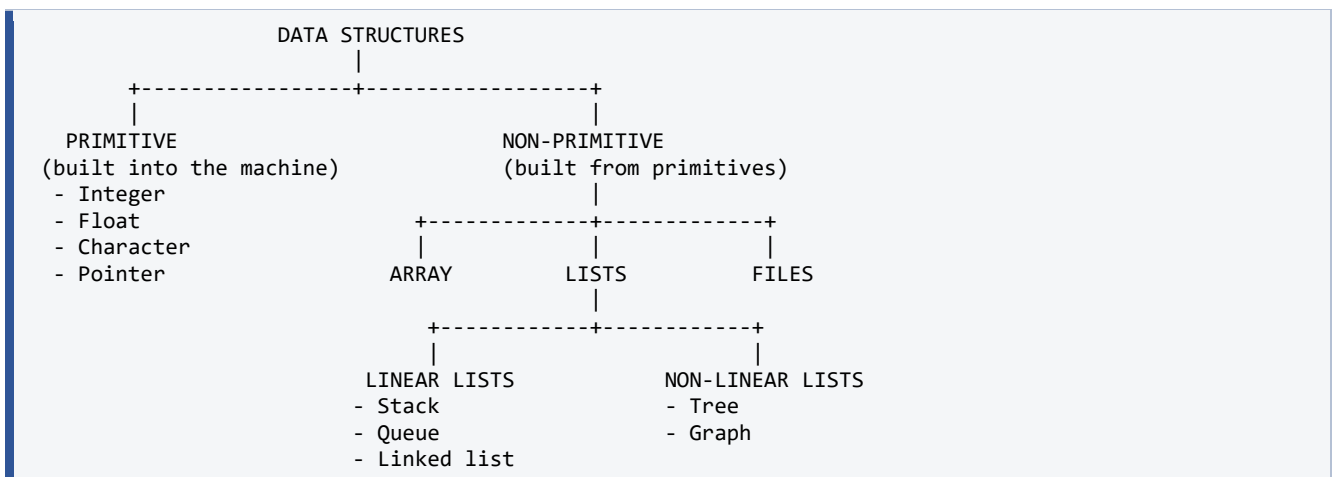
Your kitchen. Spices go in a rack, plates in a stack, groceries in a fridge with shelves. The items are the data. How you arrange them is the data structure. A good arrangement lets you find things quickly.

A data structure specifies four things:

- how the data is **organized**
- how it is **accessed**
- how the elements are **related** (degree of associativity)
- what **processing** can be done on it

Example: the marks of 5 students, [72, 85, 60, 91, 78], stored in an array. The marks are the data and the array is the structure.

1.2 Classification of Data Structures SUPPORTING



- **Primitive** structures are basic and directly handled by machine instructions (int, float, char, pointer).
- **Non-primitive** structures are built from primitive ones. Think of a LEGO model built from basic bricks.

1.3 Linear vs Non-Linear Data Structures HIGH-YIELD: Frequently Asked

Seen in: Jan-2023 Q6B (6 marks) and Jan-2025 Q1A (4 marks).

ANALOGY

Linear: elements sit one after another in a single line, like people in a queue.

Non-linear: elements connect in a branching or web-like way, like a family tree or a road map.

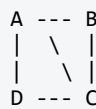
Structure	Everyday picture
Stack	Pile of plates: you take the top plate only
Queue	Line at a ticket counter: first come, first served
Tree	Family tree or folders on a computer
Graph	Road map or friendship network

LINEAR (one after another)
 Array / List : [10] -> [20] -> [30] -> [40]

NON-LINEAR (branching / network)
 TREE:



GRAPH:



Comparison table (write this in the exam)

Point	Linear Data Structure	Non-Linear Data Structure
Arrangement	Elements stored in a sequence	Elements stored hierarchically or as a network
Levels	Single level	Multiple levels
Neighbours	Each element has at most one previous and one next element	An element can connect to many elements
Traversal	Can be traversed in a single run	Cannot be traversed in a single run
Memory use	Can waste memory (arrays)	Memory is used more efficiently
Implementation	Easier	Harder
Examples	Array, Stack, Queue, Linked List	Tree, Graph

Real-world applications (Jan-2023 asked for two)

- **Stack (linear):** Undo in a text editor, the Back button in a browser. The last action is undone first.
- **Queue (linear):** a printer queue, customers at a bank counter.
- **Tree (non-linear):** folder structure on your computer, an organization chart.
- **Graph (non-linear):** Google Maps (cities are nodes, roads are edges), friends on social media.

🔑 EXAM TIP

Write 5 to 6 points in table form, then give 2 examples for each type.

1.4 Operations on Data Structures SUPPORTING

Operation	Meaning	Tiny example
Create	Reserve memory	<code>int A[5];</code>
Destroy	Free memory	<code>free(p);</code>
Selection	Access one item	<code>A[2]</code>
Update	Change an item	<code>A[2] = 99;</code>
Searching	Find an item	Find 40 in the list
Sorting	Arrange in order	[3,1,2] becomes [1,2,3]
Merging	Combine two sorted lists	[1,3] and [2,4] become [1,2,3,4]
Splitting	Break one list into many	[1,2,3,4] becomes [1,2] and [3,4]
Traversal	Visit every item once	Print all array elements

1.5 What is an Algorithm? SUPPORTING

An **algorithm** is a step-by-step procedure that solves a problem.

ANALOGY

A recipe. Follow the steps in order and you get the dish.

Five characteristics: (1) **Input:** zero or more inputs. (2) **Output:** at least one output. (3) **Finiteness:** it must stop after a finite number of steps. (4) **Definiteness:** every step is clear and unambiguous. (5) **Effectiveness:** every step is simple enough to carry out.

Example: find the largest of 3 numbers

```
1. Start
2. Read A, B, C
3. If A > B and A > C then MAX = A
   Else if B > C then MAX = B
   Else MAX = C
4. Print MAX
5. Stop
```

1.6 Time and Space Complexity ESSENTIAL FOR EXAM

There is often more than one way to solve a problem. **Complexity** tells us how much **time** and **memory** an algorithm needs as the input size **n** grows.

ANALOGY

Finding a friend's name in a phone book. Method 1: read every name from page 1 (slow). Method 2: open the middle and jump (fast). Both work, but the effort is very different.

- **Time complexity:** how many steps the algorithm takes, as a function of n .
- **Space complexity:** how much extra memory it needs.
- We count **steps, not seconds**. Seconds depend on your computer, but step counts do not.

Worked example: sum of the elements of an array

```
sum = 0;                // 1 step
for (i = 1; i <= n; i++) // about n + 1 checks
    sum = sum + A[i];    // n steps
return sum;             // 1 step

Total = 1 + (n + 1) + n + 1 = 2n + 3 --> we write O(n)
```

For large n the constants do not matter, so we keep only the fastest-growing part: **$O(n)$** , read as “order n ”.

Notation	Name	Example
$O(1)$	Constant	Access <code>A[5]</code>
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n^2)$	Quadratic	Two nested loops

1.7 Best, Average and Worst Case HIGH-YIELD: Frequently Asked

Seen in: Dec-2025 Q1A (4 marks). Linear search vs binary search was asked in Dec-2025 Q3A and Jan-2022 Q2B.

ANALOGY

You look for your keys in a row of 5 drawers.

Best case: they are in drawer 1, so you are done at once.

Worst case: they are in the last drawer or not there at all, so you open every drawer.

Average case: on average you open about half the drawers.

Linear search (the standard example)

```
Algorithm LINEAR_SEARCH(A, n, x)
1. for i = 1 to n
2.     if A[i] == x then return i      // found at position i
3. return -1                          // not found

Search for 1 in [2, 9, 3, 1, 8]
Step 1: 2 = 1 ? No      Step 2: 9 = 1 ? No      Step 3: 3 = 1 ? No
Step 4: 1 = 1 ? YES --> found at position 4 (4 comparisons)
```

Case	When it happens	Comparisons	Complexity
Best	Element is at the first position	1	$O(1)$
Average	Element is somewhere in the middle (any position equally likely)	about $(n + 1) / 2$	$O(n)$
Worst	Element is at the last position, or is absent	n	$O(n)$

MODEL ANSWER

Q: What is the meaning of the complexity of an algorithm? Explain worst, average and best case. (4 marks)

- **Complexity** measures the resources (time and memory) an algorithm uses as the input size n grows.
- **Best case:** minimum time, for the most favourable input. Linear search finds the item first, so $O(1)$.
- **Average case:** expected time over all inputs. Linear search needs about $n/2$ comparisons, so $O(n)$.
- **Worst case:** maximum time, for the least favourable input. Linear search checks every element, so $O(n)$.

Linear vs binary search (borderline: asked twice)

Binary search is not in your slides, but two papers ask you to compare it with linear search, so learn this table.

Point	Linear Search	Binary Search
Works on	Any list	Sorted list only
Method	Check one by one	Repeatedly halve the list
Best case	$O(1)$	$O(1)$
Worst case	$O(n)$	$O(\log n)$
Simplicity	Very simple	Needs sorted data

Example: for 1000 items, linear search needs up to 1000 checks and binary search only about 10. For the Jan-2022 array (15 sorted numbers, search 51), binary search is faster.

UNIT 2: Arrays

2.1 What is an Array? SUPPORTING

An **array** is a fixed-size collection of elements of the **same data type**, stored in **continuous memory locations**.

ANALOGY

A row of lockers or a train with numbered seats. Each seat has a number (the index) and all the seats are the same size.

Array A (size 5)	Index:	0	1	2	3	4
Memory address:		10	20	30	40	50
1000, 1004, 1008, ...		1000	1004	1008	1012	1016

Because memory is continuous, the computer can calculate where any element sits **without searching**. That is why array access is $O(1)$.

$$\text{Address of } A[i] = \text{Base} + w \times (i - \text{LB})$$

Base = address of the first element, w = size of one element in bytes,
LB = lower bound (first index, usually 0 or 1)

Example: Base = 1000, $w = 4$, LB = 0 $\rightarrow A[3]$ is at $1000 + 4 \times 3 = 1012$

Insertion and deletion in an array need **shifting** the elements that follow, so they cost $O(n)$. Inserting 25 at position 2 of [10, 20, 30, 40] pushes 20, 30, 40 one place right.

2.2 Two-Dimensional Arrays and Memory Order SUPPORTING

A 2-D array is a table with **rows** and **columns**. Memory is a single line, so the table must be flattened in one of two ways:

- **Row-major order:** store the first row, then the second row, and so on.
- **Column-major order:** store the first column, then the second column, and so on.

Logical table (3 x 3):	ROW-MAJOR memory:
a11 a12 a13	a11 a12 a13 a21 a22 a23 a31 a32 a33
a21 a22 a23	(row 1) (row 2) (row 3)
a31 a32 a33	
	COLUMN-MAJOR memory:
	a11 a21 a31 a12 a22 a32 a13 a23 a33
	(col 1) (col 2) (col 3)

ANALOGY

Reading a book line by line is row-major. Reading a newspaper down each column is column-major.

2.3 Address Calculation in 2-D Arrays VERY HIGH-YIELD: Asked in 4/4 papers

Seen in: Dec-2025 Q1B, Jan-2022 Q1A, Jan-2023 Q1B, Jan-2025 Q1B. Always 6 marks.

Rows $R = \text{Upper bound} - \text{Lower bound} + 1$
Columns $C = \text{Upper bound} - \text{Lower bound} + 1$

ROW-MAJOR: $\text{Address}(i, j) = \text{Base} + w \times [(i - \text{LBr}) \times C + (j - \text{LBC})]$
COLUMN-MAJOR: $\text{Address}(i, j) = \text{Base} + w \times [(j - \text{LBC}) \times R + (i - \text{LBr})]$

TIP

Row-major: skip the complete rows above you ($i - LB$), each row is C columns long, then add the steps inside your row.
Column-major: skip the complete columns to your left ($j - LB$), each column is R rows long, then add the steps inside your column.

Five-step recipe (follow it in the exam)

1. **Write the given data:** Base, w , lower bound, size, and the element asked.
2. **Find R and C .** A 4×5 array means $R = 4$ and $C = 5$. For bounds like **1..10**, $R = 10 - 1 + 1$.
3. **Write the formula** (it earns marks).
4. **Substitute** and calculate.
5. **State the result** for each order.

Worked example 1 (Jan-2025 Q1B)

A is 4×5 , Base = 2500, $w = 4$ bytes, lower bound = 1. Find the address of $A[3][4]$.

$R = 4, C = 5, LB = 1$
Row-major : $2500 + 4 \times [(3 - 1) \times 5 + (4 - 1)] = 2500 + 4 \times 13 = 2552$
Column-major: $2500 + 4 \times [(4 - 1) \times 4 + (3 - 1)] = 2500 + 4 \times 14 = 2556$

Worked example 2 (Dec-2025 Q1B)

Ary is 4×4 , Base = 4000, $w = 4$, lower bound = 0. Find the address of $Ary[3][3]$.

$R = 4, C = 4, LB = 0$
Row-major : $4000 + 4 \times [(3 - 0) \times 4 + (3 - 0)] = 4000 + 4 \times 15 = 4060$
Column-major: $4000 + 4 \times [(3 - 0) \times 4 + (3 - 0)] = 4000 + 4 \times 15 = 4060$
(Same value: the array is square and the element is on the diagonal. Still show both.)

Worked example 3 (Jan-2022 Q1A)

$arr[1..10][1..15]$, Base = 200, $w = 2$. Find $arr[8][9]$ in both orders.

$R = 10 - 1 + 1 = 10, C = 15 - 1 + 1 = 15$
Column-major: $200 + 2 \times [(9 - 1) \times 10 + (8 - 1)] = 200 + 2 \times 87 = 374$
Row-major : $200 + 2 \times [(8 - 1) \times 15 + (9 - 1)] = 200 + 2 \times 113 = 426$

Worked example 4 (Jan-2023 Q1B)

$arr[1..30][1..10]$, Base = 400, $w = 4$. Find $arr[28][5]$ in row-major and $arr[16][7]$ in column-major.

$R = 30, C = 10$
Row-major, $arr[28][5]$: $400 + 4 \times [(28 - 1) \times 10 + (5 - 1)] = 400 + 4 \times 274 = 1496$
Column-major, $arr[16][7]$: $400 + 4 \times [(7 - 1) \times 30 + (16 - 1)] = 400 + 4 \times 195 = 1180$

COMMON MISTAKES

- Using the wrong count: row-major multiplies by the COLUMNS, column-major multiplies by the ROWS.
- Forgetting to subtract the lower bound when it is 1.
- Forgetting to multiply by the element size w .
- Computing R and C wrongly when bounds are written as 1..10.

ANSWER

Practice 1: $A[1..5][1..6]$, Base 1000, $w = 2$. Find $A[4][3]$ in both orders. Answer: row-major 1040, column-major 1026.

Practice 2: $B[0..3][0..4]$, Base 500, $w = 8$. Find $B[2][3]$ in both orders. Answer: row-major 604, column-major 612.

2.4 Polynomial Representation HIGH-YIELD: Frequently Asked

Seen in: Dec-2025 Q1C and Jan-2025 Q1C (6 marks each).

A polynomial such as $25X^3Y^2 + 30X^2Y + 4X + 3$ is a list of **terms**. Each term has a **coefficient** and **powers** of the variables.

ANALOGY

A shopping list. Each line has a quantity (the coefficient) and an item with its attributes (the powers).

Method 1: array (in your slides). For one variable, store each coefficient at the index equal to its power. This wastes space when many terms are zero.

$5X^3 + 2X + 7$ is stored as: Index (power): 0 1 2 3
Coefficient : [7] [2] [0] [5]

Method 2: singly linked list (what the exam asks). Every term becomes one node, and the nodes are chained with pointers. Only non-zero terms are stored, so no space is wasted.

Node format

COEFF	X-power	Y-power	Z-power	LINK
For a polynomial in X and Y only, drop pz.				

```

struct node {
    int coeff;
    int px, py, pz;
    struct node *link;
};
    
```

Example (Dec-2025): $43X^3Y^2Z^3 + 50X^2Y^2Z + 15XYZ + 15X + 40Y^2 + 8Z + 6$

Term	Coeff	X	Y	Z
$43X^3Y^2Z^3$	43	3	2	3
$50X^2Y^2Z$	50	2	2	1
$15XYZ$	15	1	1	1
$15X$	15	1	0	0
$40Y^2$	40	0	2	0
$8Z$	8	0	0	1
6	6	0	0	0

```

FIRST
|
v
+-----+-----+-----+-----+-----+
| 43 | 3 | 2 | 3 | o-----> | 50 | 2 | 2 | 1 | o-----> ... -> | 6 | 0 | 0 | 0 | NULL |
+-----+-----+-----+-----+-----+
(7 nodes in total, one per term. The last node's LINK is NULL.)
    
```

Jan-2025 polynomial: $25X^3Y^2 + 30X^2Y + 14XY^4 + 4X + 8Y + 3$ has 6 terms: $(25, 3, 2) \rightarrow (30, 2, 1) \rightarrow (14, 1, 4) \rightarrow (4, 1, 0) \rightarrow (8, 0, 1) \rightarrow (3, 0, 0) \rightarrow \text{NULL}$.

Algorithm: count the number of terms (Dec-2025)

The number of terms equals the number of nodes, so traverse and count.

```

Algorithm COUNT_TERMS(FIRST)
1. COUNT = 0
2. TEMP = FIRST
3. Repeat steps 4 and 5 while TEMP != NULL
4.     COUNT = COUNT + 1
5.     TEMP = LINK(TEMP)
6. Return COUNT
    (for the Dec-2025 polynomial the answer is 7)
    
```

Algorithm: display the polynomial (Jan-2025)

```
Algorithm DISPLAY(FIRST)
1. If FIRST = NULL then print "Polynomial is empty" and Return
2. TEMP = FIRST
3. Repeat while TEMP != NULL
4.   Print COEFF(TEMP)
5.   If PX(TEMP) > 0 then print "X^", PX(TEMP)
6.   If PY(TEMP) > 0 then print "Y^", PY(TEMP)
7.   If LINK(TEMP) != NULL then print " + "
8.   TEMP = LINK(TEMP)
9. Stop
```

MODEL ANSWER

6-mark answer layout: (1) linked-list diagram, 2 marks; (2) node format with field names, 2 marks; (3) the algorithm, 2 marks.

2.5 Sparse Matrix LOW PRIORITY: Never asked so far

A matrix is **sparse** if **most of its elements are zero**.

ANALOGY

A cinema with only 3 people in 100 seats. Writing down all 100 seats is a waste. It is better to note "seat 14, seat 52, seat 77".

Triplet representation: store only the non-zero elements as (Row, Column, Value). The first row is a header with total rows, total columns and the number of non-zero values.

Matrix (3 x 4):
0 0 5 0
0 0 0 0
3 0 0 7

Triplet table:

Row	Col	Value
3	4	3
1	3	5
3	1	3
3	4	7

<- header: 3 rows, 4 cols, 3 non-zero

The 12 stored values shrink to 4 rows of 3 numbers.

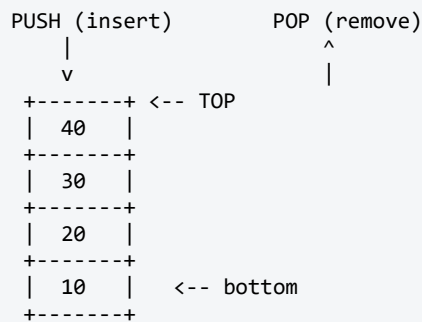
UNIT 3: Stack

3.1 What is a Stack?

A **stack** is a linear list in which insertion and deletion happen at **one end only**, called the **TOP**. It follows **LIFO**: Last In, First Out.

ANALOGY

A pile of plates. You put a new plate on top and you take a plate from the top. The plate you placed last is the first one you remove.



- **Overflow**: trying to PUSH when the stack is full.
- **Underflow**: trying to POP when the stack is empty.
- Using an array `S[1..N]` and a pointer `TOP`. When the stack is empty, `TOP = 0`.

Example: PUSH 10, PUSH 20, PUSH 30, POP, PUSH 40.

Operation	Stack (bottom -> top)	TOP	Returned
PUSH 10	10	1	
PUSH 20	10 20	2	
PUSH 30	10 20 30	3	
POP	10 20	2	30
PUSH 40	10 20 40	3	

Stack algorithms (never asked so far, but they are the base of everything below)

<pre>Procedure PUSH(S, TOP, X) 1. [Overflow?] If TOP >= N then write "Stack Overflow"; Return 2. [Increase TOP] TOP = TOP + 1 3. [Insert] S[TOP] = X 4. Return</pre>	<pre>Function POP(S, TOP) 1. [Underflow?] If TOP = 0 then write "Stack Underflow"; Return 2. [Remove] X = S[TOP] TOP = TOP - 1 3. Return X</pre>
<pre>Function PEEP(S, TOP, I) (value of the Ith item from the top) 1. If TOP - I + 1 <= 0 then Underflow 2. Return S[TOP - I + 1]</pre>	<pre>Procedure CHANGE(S, TOP, X, I) (replace the Ith item from the top) 1. If TOP - I + 1 <= 0 then Underflow 2. S[TOP - I + 1] = X</pre>

- **Applications of stack**: undo/redo, browser Back button, function calls and recursion, checking balanced brackets, and **evaluating expressions** (the next sections).

3.2 Polish Notation: Infix, Prefix, Postfix

There are three ways to write an expression, depending on where the operator goes.

Name	Operator position	Example	Used by
Infix	Between the operands	A + B	Humans
Prefix (Polish)	Before the operands	+ A B	Some compilers

Name	Operator position	Example	Used by
Postfix (Reverse Polish)	After the operands	A B +	Computers, using a stack

ANALOGY

Infix needs brackets and rules ("multiply before add"). Postfix needs neither. It is like a calculator that takes the numbers first and the operation last: press 5, press 3, press +. The order of the symbols already says what to do first.

Infix	Postfix	Prefix
A + B	A B +	+ A B
A + B * C	A B C * +	+ A * B C
(A + B) * C	A B + C *	* + A B C
A * B + C / D	A B * C D / +	+ * A B / C D

Operator precedence

Priority	Operators	Associativity
Highest	$\wedge$ (power)	Right to left
Middle	* /	Left to right
Lowest	+ -	Left to right

3.3 Infix to Postfix Conversion VERY HIGH-YIELD: Asked in 4/4 papers

Seen in: Dec-2025 Q2A, Jan-2023 Q2B, Jan-2025 Q2A (6 to 8 marks).

The rules (scan the infix expression left to right, one symbol at a time)

- Operand** (number or letter): send it straight to the **output**.
- Opening bracket (**: push it on the stack.
- Closing bracket)**: pop operators to the output until you meet **(**. Remove the **(** and do not output it.
- Operator**: while the operator on the top of the stack has **higher or equal precedence**, pop it to the output. Then push the new operator. (For $\wedge$, pop only if the top has strictly higher precedence, because $\wedge$ goes right to left.)
- End of expression**: pop everything left on the stack to the output.

ANALOGY

The stack is a waiting room for operators. An operator must wait until everything that has to happen before it has been written out. Brackets build a small private waiting room.

Example 1 (Dec-2025 Q2A): $2 + 3 * (9 - 4) / 5$

Symbol	Stack (bottom to top)	Output (postfix so far)	Action
2		2	operand
+	+	2	push
3	+	2 3	operand
*	+ *	2 3	* beats +, push
(	+ * (	2 3	push (
9	+ * (	2 3 9	operand
-	+ * (-	2 3 9	top is (, push
4	+ * (-	2 3 9 4	operand
)	+ *	2 3 9 4 -	pop until (

Symbol	Stack (bottom to top)	Output (postfix so far)	Action
/	+ /	2 3 9 4 - *	pop * (equal), push /
5	+ /	2 3 9 4 - * 5	operand
end		2 3 9 4 - * 5 / +	pop all

Postfix2 3 9 4 - * 5 / +

Example 2 (Jan-2025 Q2A): 5 + 3 - (4 - 2) * 9 / 6

Symbol	Stack	Output	Action
5		5	operand
+	+	5	push
3	+	5 3	operand
-	-	5 3 +	pop + (equal), push -
(	- (	5 3 +	push (
4	- (	5 3 + 4	operand
-	- (-	5 3 + 4	top is (, push
2	- (-	5 3 + 4 2	operand
)	-	5 3 + 4 2 -	pop until (
*	- *	5 3 + 4 2 -	* beats -, push
9	- *	5 3 + 4 2 - 9	operand
/	- /	5 3 + 4 2 - 9 *	pop * (equal), push /
6	- /	5 3 + 4 2 - 9 * 6	operand
end		5 3 + 4 2 - 9 * 6 / -	pop all

Postfix5 3 + 4 2 - 9 * 6 / -

Example 3 (Jan-2023 Q2B): 8 + 4 / 2 * 3 - 6 + 7 - 8 ^ 1

Symbol	Stack	Output	Action
8		8	operand
+	+	8	push
4	+	8 4	operand
/	+ /	8 4	/ beats +, push
2	+ /	8 4 2	operand
*	+ *	8 4 2 /	pop / (equal), push *
3	+ *	8 4 2 / 3	operand
-	-	8 4 2 / 3 * +	pop *, pop +, push -
6	-	8 4 2 / 3 * + 6	operand
+	+	8 4 2 / 3 * + 6 -	pop -, push +
7	+	8 4 2 / 3 * + 6 - 7	operand
-	-	8 4 2 / 3 * + 6 - 7 +	pop +, push -
8	-	8 4 2 / 3 * + 6 - 7 + 8	operand
^	- ^	8 4 2 / 3 * + 6 - 7 + 8	^ beats -, push
1	- ^	... 7 + 8 1	operand
end		8 4 2 / 3 * + 6 - 7 + 8 1 ^ -	pop all

Postfix8 4 2 / 3 * + 6 - 7 + 8 1 ^ -

⚠ COMMON MISTAKES

- Forgetting to pop on EQUAL precedence (+ then - must pop the first one).
- Writing the bracket in the output. Brackets never appear in postfix.
- Forgetting to empty the stack at the end.
- Not showing the stack column. The table is where the marks are.

✓ ANSWER

Practice (convert):

1. $A + B * C - D \rightarrow A B C * + D -$
2. $(A + B) * C / D \rightarrow A B + C * D /$
3. $A * (B + C) - D / E \rightarrow A B C + * D E / -$

3.4 Evaluation of a Postfix Expression 🔥 VERY HIGH-YIELD: Asked in 4/4 papers

Seen in: Jan-2022 Q1B, Jan-2023 Q3B, and together with conversion in Dec-2025 Q2A and Jan-2025 Q2A.

The rules (scan the postfix expression left to right)

1. **Operand:** push it on the stack.
2. **Operator:** pop the top two values. The **first pop is B (right operand)** and the **second pop is A (left operand)**. Compute **A operator B** and push the result back.
3. **End:** the single value left on the stack is the answer.

⚠ COMMON MISTAKES

Order matters for -, / and ^. If the stack holds ... 9 4 and the operator is -, the answer is $9 - 4 = 5$, not $4 - 9$.

Algorithm EVALUATE_POSTFIX(POSTFIX)

1. TOP = 0
2. Repeat steps 3 to 5 for each symbol X, scanning left to right
3. If X is an operand then PUSH(X)
4. Else B = POP(), A = POP()
5. PUSH(A X B) // apply the operator X
6. Return POP() // final answer

Example 1 (Jan-2022 Q1B): $M = 8\ 2\ 3\ ^\wedge / 2\ 3\ * + 5\ 1\ * -$

Symbol	Action	Stack after the step (bottom to top)
8	push	8
2	push	8 2
3	push	8 2 3
^	pop 3 and 2; $2^3 = 8$	8 8
/	pop 8 and 8; $8/8 = 1$	1
2	push	1 2
3	push	1 2 3
*	pop 3 and 2; $2*3 = 6$	1 6
+	pop 6 and 1; $1+6 = 7$	7
5	push	7 5
1	push	7 5 1
*	pop 1 and 5; $5*1 = 5$	7 5
-	pop 5 and 7; $7-5 = 2$	2

Result = 2

Example 2 (Jan-2023 Q3B): 8 7 6 3 / 4 2 // + /

Symbol	Action	Stack after the step
8	push	8
7	push	8 7
6	push	8 7 6
3	push	8 7 6 3
/	6/3 = 2	8 7 2
4	push	8 7 2 4
2	push	8 7 2 4 2
/	4/2 = 2	8 7 2 2
/	2/2 = 1	8 7 1
+	7+1 = 8	8 8
/	8/8 = 1	1

Result = 1

Example 3 (Dec-2025 Q2A): evaluate 2 3 9 4 - * 5 / +

2 -> [2]	3 -> [2 3]	9 -> [2 3 9]	4 -> [2 3 9 4]
- : 9 - 4 = 5	-> [2 3 5]		
* : 3 * 5 = 15	-> [2 15]		
5 -> [2 15 5]			
/ : 15 / 5 = 3	-> [2 3]		
+ : 2 + 3 = 5	-> [5]		
			RESULT = 5

Example 4 (Jan-2025 Q2A): evaluate 5 3 + 4 2 - 9 * 6 / -

5 -> [5]	3 -> [5 3]		
+ : 5 + 3 = 8	-> [8]		
4 -> [8 4]	2 -> [8 4 2]		
- : 4 - 2 = 2	-> [8 2]		
9 -> [8 2 9]			
* : 2 * 9 = 18	-> [8 18]		
6 -> [8 18 6]			
/ : 18 / 6 = 3	-> [8 3]		
- : 8 - 3 = 5	-> [5]		
			RESULT = 5

☒ ANSWER

Practice (evaluate):

1. 5 4 6 + * 4 9 3 / + * → 350

(4+6=10; 5*10=50; 9/3=3; 4+3=7; 50*7=350)

2. 7 5 2 + * → 49

3. 5 6 2 - + → 9

 **EXAM TIP**

In the exam: write the rules in one line, draw the table (symbol, action, stack), and box the final answer.

3.5 Infix to Prefix and Prefix Evaluation 🚩 LOW PRIORITY: Never asked so far

Quick manual method: fully bracket the expression using precedence, then move every operator **just before** its two operands and remove the brackets.

A + B * C	--> A + (B * C)	--> + A * B C
(A + B) * C	--> ((A + B) * C)	--> * + A B C

Evaluating prefix: scan **right to left**. Push operands. At an operator, pop the first value as **A** and the second as **B**, compute **A operator B** and push.

```
Evaluate + 5 * 3 4      (scan from the right)
4 -> [4]      3 -> [4 3]
* : pop 3 (A), pop 4 (B) -> 3 * 4 = 12 -> [12]
5 -> [12 5]
+ : pop 5 (A), pop 12 (B) -> 5 + 12 = 17 -> [17]      RESULT = 17
```

3.6 Recursion LOW PRIORITY: Never asked so far

Recursion means a function **calls itself** to solve a smaller version of the same problem. Every recursive function needs a **base case** (when to stop) and a **recursive case** (the smaller call).

ANALOGY

Russian nesting dolls. To find the smallest doll you open a doll, find a smaller one inside, and repeat until you reach the doll that cannot be opened. That last doll is the base case.

```
int fact(int n) {
    if (n == 0 || n == 1)    // base case
        return 1;
    return n * fact(n - 1);  // recursive case
}

fact(3) = 3 * fact(2)
        = 3 * (2 * fact(1))
        = 3 * (2 * 1)
        = 6
```

Call stack (one frame per call):

fact(1)	n=1	returns 1
fact(2)	n=2	returns 2 * 1 = 2
fact(3)	n=3	returns 3 * 2 = 6

The system uses a **stack** to remember each pending call. That is why recursion belongs in the stack unit. Without a base case the stack overflows.

Tower of Hanoi: to move n disks from A to C using B: move $n-1$ disks $A \rightarrow B$, move the largest disk $A \rightarrow C$, then move $n-1$ disks $B \rightarrow C$. Number of moves = $2^n - 1$ ($n = 3$ needs 7 moves).

3.7 Recognize a String Using a Stack LOW PRIORITY: Never asked so far

Problem (in your slides): check whether a string has the form **w c w^R**, a word, then the letter **c**, then the same word reversed. Example: **abcba** is valid, **abcab** is not.

Algorithm RECOGNIZE(string)

1. Read characters left to right and PUSH each one until the letter 'c' is read.
(If the string ends without a 'c', it is INVALID.)
2. Read the next character X and POP the top element Y.
If X is not equal to Y then the string is INVALID. Stop.
3. Repeat step 2 until the string ends.
4. If the stack is empty at the end, the string is VALID, otherwise INVALID.

Trace for a b c b a :

push a, push b, see c	stack [a b]	
read b, pop b (match)	stack [a]	
read a, pop a (match)	stack []	--> VALID

Second form in your slides: **a^ba**. PUSH every **a**. For every **b**, POP one **a**. The string is valid if the stack is empty at the end and no **a** comes after a **b**. Example: **aaabbb** is valid, **aabbb** is not.

Quick revision box: Unit 3

Topic	Remember
Stack	LIFO. PUSH and POP at TOP. Empty when TOP = 0.

Topic	Remember
Infix to postfix	Operand to output. (push.) pop to (. Operator pops higher or equal, then push. End: pop all.
Postfix evaluation	Operand push. Operator: pop B, pop A, push A op B. Answer = last value.
Prefix evaluation	Scan right to left. First pop is A, second is B.
Recursion	Base case + recursive case. Uses the call stack.

UNIT 4: Queue

4.1 What is a Queue? (Simple Queue) HIGH-YIELD: Frequently Asked

Seen in: Dec-2025 Q2C (patient registration) and Jan-2025 Q2B (insert in a simple queue using static memory).

A **queue** is a linear list in which **insertion happens at one end (REAR)** and **deletion happens at the other end (FRONT)**. It follows **FIFO**: First In, First Out.

 ANALOGY
A line at a ticket counter. New people join at the back, and the person who came first is served first. Nobody can jump in the middle.

A line at a ticket counter. New people join at the back, and the person who came first is served first. Nobody can jump in the middle.

```

DEQUEUE (remove)                                ENQUEUE (insert)
<----- FRONT [ 10 | 20 | 30 | 40 ] REAR <-----

Array view (N = 4):      Index:   1       2       3       4
                        +-----+-----+-----+-----+
                        | 10 | 20 | 30 |   |
                        +-----+-----+-----+-----+
                        ^               ^
                        F               R

```

- **F (front)** points to the first element. **R (rear)** points to the last element.
- **Empty queue:** $F = 0$ and $R = 0$.
- **Overflow:** insert into a full queue. **Underflow:** delete from an empty queue.
- **Applications:** printer queue, CPU scheduling, customers at a service counter, BFS in graphs, call centres.

Algorithms for a simple queue using an array (static memory)

<pre> Procedure ENQUEUE(Q, F, R, N, Y) (insert Y at the rear) 1. [Overflow?] If R >= N then write "Queue Overflow"; Return 2. [Move rear] R = R + 1 3. [Insert] Q[R] = Y 4. [Set front] If F = 0 then F = 1 5. Return </pre>	<pre> Function DEQUEUE(Q, F, R) (delete and return the front item) 1. [Underflow?] If F = 0 then write "Queue Underflow"; Return 2. [Delete] Y = Q[F] 3. [Queue now empty?] If F = R then F = 0 and R = 0 Else F = F + 1 4. Return Y </pre>
--	--

Example (N = 4):

Operation	F	R	Queue (slots 1 to 4)	Remark
start	0	0	_ _ _ _	empty
ENQUEUE 10	1	1	10 _ _ _	
ENQUEUE 20	1	2	10 20 _ _	
ENQUEUE 30	1	3	10 20 30 _	
DEQUEUE	2	3	_ 20 30 _	returns 10
ENQUEUE 40	2	4	_ 20 30 40	R = N now
ENQUEUE 50	2	4	_ 20 30 40	OVERFLOW, but slot 1 is free!

That last row is the weakness of a simple queue. It leads straight to the circular queue.

4.2 Circular Queue HIGH-YIELD: Frequently Asked

Seen in: Jan-2023 Q2A (10 marks, trace with front and rear) and Jan-2025 Q2B (justify, 6 marks).

ANALOGY

A round conveyor belt at a sushi restaurant. When the belt reaches the end of the line it loops back to the start, so the plates always reuse the same free space.

In a **circular queue** the last slot is joined to the first slot, so after position N the rear **wraps around** to position 1. Space freed by deletions is reused.

SIMPLE queue (N = 4) after: insert 10,20,30,40 ; delete ; delete

```

Index:   1    2    3    4
+-----+-----+-----+
| - | - | 30 | 40 |      F = 3,  R = 4 = N
+-----+-----+-----+

```

insert(50) --> "Overflow!" although slots 1 and 2 are EMPTY (wasted).

CIRCULAR queue, same operations: R wraps from 4 to 1, so 50 is stored in slot 1.

```

+-----+-----+-----+
| 50 | - | 30 | 40 |      F = 3,  R = 1
+-----+-----+-----+

```

Algorithms (array of size N, empty when F = 0)

Procedure CQINSERT(Q, F, R, N, Y)

1. [Queue full?]
 - If (F = 1 and R = N) or (F = R + 1) then write "Overflow"; Return
2. [Move rear with wrap-around]
 - If R = N then R = 1
 - Else R = R + 1
3. [Insert] Q[R] = Y
4. [Set front] If F = 0 then F = 1
5. Return

Function CQDELETE(Q, F, R, N)

1. [Underflow?]
 - If F = 0 then write "Underflow"; Return
2. [Delete] Y = Q[F]
3. [Move front]
 - If F = R then F = 0 and R = 0 (now empty)
 - Else if F = N then F = 1
 - Else F = F + 1
4. Return Y

Your lecture slides show the same procedures (CQINSERT and CQDELETE) with the full check written as "F = R after moving R". Both behave identically, so use the version your faculty taught.

Trace for Jan-2023 Q2A (circular queue of size 6)

Operations: insert(A), insert(B), insert(C), insert(D), insert(E), delete(), delete(), insert(F), insert(G), insert(H), insert(I), delete(), delete(), delete().

Operation	F	R	Queue (slots 1 to 6)	Remark
start	0	0	_ _ _ _ _ _	empty
insert(A)	1	1	A _ _ _ _ _	
insert(B)	1	2	A B _ _ _ _	
insert(C)	1	3	A B C _ _ _	
insert(D)	1	4	A B C D _ _	
insert(E)	1	5	A B C D E _	
delete()	2	5	_ B C D E _	removes A
delete()	3	5	_ _ C D E _	removes B
insert(F)	3	6	_ _ C D E F	
insert(G)	3	1	G _ C D E F	R wraps to 1
insert(H)	3	2	G H C D E F	queue is FULL
insert(I)	3	2	G H C D E F	OVERFLOW, I is rejected
delete()	4	2	G H _ D E F	removes C
delete()	5	2	G H _ _ E F	removes D
delete()	6	2	G H _ _ _ F	removes E

Final state: F = 6, R = 2. The queue holds F (slot 6), then G (slot 1), then H (slot 2). If your faculty numbers slots from 0, subtract 1 from every F and R.

 **EXAM TIP**

The trick in this question is insert(I): the queue is already full after insert(H), so the answer must show an overflow. State clearly which convention you use (F = R = 0 means empty).

Justify: “A circular queue is more efficient than a simple queue under static allocation” (Jan-2025 Q2B)

 **MODEL ANSWER**

- **Static allocation** means the array size is fixed at compile time, so the memory cannot grow.
- In a **simple queue**, deletions only move the front forward. The rear can never go back, so slots freed at the front are **wasted**. The queue reports overflow even when free slots exist (see the N = 4 example above).
- In a **circular queue**, the rear **wraps around** to slot 1, so every freed slot is reused and the fixed memory is used fully.
- **Example:** size 4, insert 10, 20, 30, 40, delete twice, insert 50. The simple queue overflows. The circular queue stores 50 in slot 1.
- Both queues have O(1) insert and delete, but the circular queue never wastes the fixed memory.

Then write the **ENQUEUE algorithm for the simple queue** from section 4.1, which the same question asks for.

4.3 Design Question: Patient Registration System (static memory)

 HIGH-YIELD: Frequently Asked

Seen in: Dec-2025 Q2C (6 marks, BL6): add new patients, call the next patient, view the waiting list.

Step 1: choose the data structure. Patients must be served in the order they arrive (fair, first come first served), so use a **queue**. Because the question says **static memory**, store it in an **array of fixed size MAX**.

Step 2: map each feature to a queue operation.

Feature	Queue operation	Cost
Add a new patient	ENQUEUE at the rear	O(1)
Call the next patient	DEQUEUE from the front	O(1)
View the waiting list	Traverse from F to R (no change to the queue)	O(n)

Algorithm ADD_PATIENT(P)

1. If R = MAX then
 write "Waiting list full"; Stop

2. R = R + 1

3. Q[R] = P

4. If F = 0 then F = 1

5. Stop

Algorithm CALL_NEXT()

1. If F = 0 then
 write "No patient waiting"; Stop

2. P = Q[F]

3. Write "Call patient", P

4. If F = R then F = 0 and R = 0
 Else F = F + 1

5. Stop

Algorithm VIEW_WAITING()

1. If F = 0 then write "No patient waiting"; Stop

2. For I = F to R

3. Write Q[I] // patients in the order they will be called

4. Stop

Example run (MAX = 5): add Asha, add Ravi, add Meena → list shows Asha, Ravi, Meena. Call next → Asha is called. View list → Ravi, Meena.

Optional C code (0-based indexing, front = -1 means empty)

```
#define MAX 5
struct patient { int id; char name[20]; };
struct patient q[MAX];
int front = -1, rear = -1;

void addPatient(struct patient p) {
    if (rear == MAX - 1) { printf("Waiting list full\n"); return; }
    if (front == -1) front = 0;
    q[++rear] = p;
}
void callNext() {
    if (front == -1) { printf("No patient waiting\n"); return; }
    printf("Next: %s\n", q[front].name);
    if (front == rear) front = rear = -1; else front++;
}
void viewWaiting() {
    if (front == -1) { printf("No patient waiting\n"); return; }
    for (int i = front; i <= rear; i++) printf("%d %s\n", q[i].id, q[i].name);
}
```

TIP

To earn the BL6 “justify your design” marks, add one line: “A circular queue would avoid wasting slots after many patients have been called.”

4.4 Priority Queue ESSENTIAL FOR EXAM

Seen in: Jan-2022 Q1C (OR option, 6 marks): implement a priority queue using a 2-D array.

In a **priority queue**, every item has a **priority**. The item with the **highest priority is served first**. Items with the same priority are served in FIFO order.

ANALOGY

A hospital emergency room. A patient with a heart attack is treated before a patient with a sprained ankle, even if the ankle patient arrived earlier.

2-D array implementation: use **one row per priority level**. Each row is a small queue with its own front and rear pointers. Row 1 has the highest priority.

PQ[M][N] (M priority rows, each row a queue of N slots) F[i], R[i] = pointers of row i

Priority	F	R	Slots
Row 1 (High)	1	2	Fire Accident -
Row 2 (Med)	1	1	Fever - -
Row 3 (Low)	0	0	- - -

INSERT : go to the row of the item's priority, add at that row's REAR.

DELETE : scan rows 1, 2, 3 ...; serve the FRONT of the first non-empty row.
Here the next item served is "Fire".

```

Procedure PQ_INSERT(PQ, F, R, N, X, P)          // X = item, P = its priority (row)
1. If R[P] = N then write "Row P is full (Overflow)"; Return
2. R[P] = R[P] + 1
3. PQ[P][R[P]] = X
4. If F[P] = 0 then F[P] = 1
5. Return

Function PQ_DELETE(PQ, F, R, M)
1. For I = 1 to M                               // highest priority first
2.   If F[I] != 0 then
3.     Y = PQ[I][F[I]]
4.     If F[I] = R[I] then F[I] = 0 and R[I] = 0
5.     Else F[I] = F[I] + 1
6.   Return Y
7. Write "Queue Underflow: all rows are empty"
   (Initially F[I] = 0 and R[I] = 0 for every row.)

```

Quick revision box: Unit 4

Topic	Remember
Queue	FIFO. Insert at REAR, delete at FRONT. Empty when F = 0.
Simple queue problem	Rear never moves back, so freed front slots are wasted (false overflow).
Circular queue	Rear and front wrap from N to 1. Full when F = R + 1 (or F = 1 and R = N).
Trace answers	Show F, R and the whole array after every operation. State your convention.
Design questions	Choose the structure, state static or dynamic, then give ADD, CALL and VIEW algorithms.
Priority queue	One row per priority. Delete from the first non-empty row.

☒ ANSWER

Practice (circular queue of size 5): insert(10), insert(20), insert(30), delete(), insert(40), insert(50), insert(60), insert(70).

Answer: after insert(60) the queue holds 20, 30, 40, 50, 60 with F = 2 and R = 1 (R wrapped), so the queue is full. insert(70) gives OVERFLOW. Final F = 2, R = 1.

UNIT 5: Linked Lists

5.1 Why Linked Lists? (Array vs Linked List) SUPPORTING

An array has a **fixed size** and needs shifting for insertion and deletion. A **linked list** stores each item in a separate **node** that also holds the **address of the next node**, so the list can grow and shrink freely.

ANALOGY

A treasure hunt. Each clue (node) holds a prize (the data) and tells you where the next clue is hidden (the pointer). You do not need all the clues in one place, you only need to know where the first clue is.

Point	Array	Linked List
Size	Fixed at compile time	Grows and shrinks at run time
Memory	Continuous block	Scattered nodes joined by pointers
Access to the i-th item	Direct, $O(1)$	Must walk from the start, $O(n)$
Insertion / deletion	Shifting needed, $O(n)$	Only pointers change, $O(1)$ once the position is known
Extra memory	None	One pointer per node
Memory waste	Unused slots are wasted	Only the memory needed is used

5.2 Static vs Dynamic Memory Allocation (malloc) HIGH-YIELD: Frequently Asked

Seen in: Dec-2025 Q2B (justify “a list created with dynamic allocation is flexible in size”, BL5) and Dec-2025 Q2C / Jan-2025 Q2B (static memory).

Point	Static allocation	Dynamic allocation
When memory is fixed	Compile time	Run time, on demand
Size	Fixed, cannot change	Can grow and shrink
Typical structure	Array	Linked list
Risk	Waste if too big, overflow if too small	Little waste. Must free the memory yourself
C tool	Normal declaration <code>int A[100];</code>	<code>malloc()</code> to get memory, <code>free()</code> to give it back

```
struct node *NEW = (struct node*) malloc(sizeof(struct node)); // ask for ONE node
if (NEW == NULL) { /* memory not available */ }
...
free(NEW); // give the node back
```

MODEL ANSWER

Justify: “The list prepared using dynamic memory allocation is flexible in size.”

- With dynamic allocation each node is requested with `malloc()` only when it is needed, and returned with `free()` when it is deleted.
- So the list grows by exactly one node per new item and shrinks by one per deletion. There is **no fixed limit** and **no unused space**.
- **Example:** a customer list. An array of 100 customers wastes 97 slots if only 3 customers arrive, and fails when the 101st arrives. A dynamic list simply allocates a new node for the 101st, the 102nd and so on.
- **Adding a customer:** malloc a node, fill it, and link it at the end. Deleting a customer: unlink the node and free it.

5.3 Singly Linked List HIGH-YIELD: Frequently Asked

Seen in: Dec-2025 Q2B (add at end) and Q1C (count nodes), Jan-2025 Q2C (insert at front, delete from last) and Q1C (display).

In a **singly linked list** every node has two parts: **INFO** (the data) and **LINK** (the address of the next node). The pointer **FIRST** holds the address of the first node. The LINK of the last node is **NULL**.

```

FIRST
|
v
+-----+-----+ +-----+-----+ +-----+-----+
| 10 | o----->| 20 | o----->| 30 | NULL |
+-----+-----+ +-----+-----+ +-----+-----+
  INFO   LINK

struct node {
    int info;
    struct node *link;
};

```

ANALOGY

A train. Each coach (node) carries passengers (info) and is hooked to the next coach (link). The engine (FIRST) knows only the first coach. To reach coach 5 you must walk through coaches 1 to 4.

A. Insert a node at the FRONT (Jan-2025 Q2C)

```

Function INSERT(X, FIRST)
1. NEW = (node*) malloc(sizeof(node))
2. If NEW = NULL then write "Overflow: no memory"; Return FIRST
3. INFO(NEW) = X
4. LINK(NEW) = FIRST           // new node points to the old first node
5. FIRST = NEW                 // new node becomes the first node
6. Return FIRST

BEFORE:  FIRST --> [10|o]--> [20|o]--> [30|NULL]
STEP 4:  NEW:  [5|o]--> [10|o]--> [20|o]--> [30|NULL]
AFTER :  FIRST --> [5|o]--> [10|o]--> [20|o]--> [30|NULL]      (insert 5)

```

B. Insert a node at the END (Dec-2025 Q2B: add a customer at the end)

```

Function INSEND(X, FIRST)
1. NEW = (node*) malloc(sizeof(node))
2. If NEW = NULL then write "Overflow: no memory"; Return FIRST
3. INFO(NEW) = X
4. LINK(NEW) = NULL           // it will be the last node
5. If FIRST = NULL then       // empty list: the new node is the only node
    FIRST = NEW; Return FIRST
6. SAVE = FIRST
7. Repeat while LINK(SAVE) != NULL
    SAVE = LINK(SAVE)         // walk to the last node
8. LINK(SAVE) = NEW           // hook the new node after it
9. Return FIRST

BEFORE:  FIRST --> [10|o]--> [20|o]--> [30|NULL]
                ^ SAVE stops here (its LINK is NULL)
AFTER :  FIRST --> [10|o]--> [20|o]--> [30|o]--> [40|NULL]      (insert 40)

```

C. Delete the node at the END (Jan-2025 Q2C)

```
Function DELETE_LAST(FIRST)
1. If FIRST = NULL then write "Underflow: list is empty"; Return FIRST
2. If LINK(FIRST) = NULL then          // only one node
   free(FIRST); FIRST = NULL; Return FIRST
3. PREV = FIRST; CUR = LINK(FIRST)
4. Repeat while LINK(CUR) != NULL      // find the last node and the one before it
   PREV = CUR; CUR = LINK(CUR)
5. LINK(PREV) = NULL                  // second-last node becomes the last
6. free(CUR)
7. Return FIRST

BEFORE:  FIRST --> [10|o]--> [20|o]--> [30|NULL]
          PREV      CUR
AFTER :  FIRST --> [10|o]--> [20|NULL]          (30 is freed)
```

D. Delete the node at the FRONT, count nodes, display

<pre>Function DELETE_FIRST(FIRST) 1. If FIRST = NULL then Underflow 2. TEMP = FIRST 3. FIRST = LINK(FIRST) 4. free(TEMP) 5. Return FIRST</pre>	<pre>Function COUNT_NODES(FIRST) 1. COUNT = 0; TEMP = FIRST 2. Repeat while TEMP != NULL COUNT = COUNT + 1 TEMP = LINK(TEMP) 3. Return COUNT</pre>
--	--


```
Procedure DISPLAY(FIRST)
1. TEMP = FIRST
2. Repeat while TEMP != NULL: print INFO(TEMP); TEMP = LINK(TEMP)
3. Print "NULL"
```

Working C code for all of the above

```
struct node { int info; struct node *link; };
struct node *first = NULL;

void insertFront(int x) {
    struct node *n = (struct node*) malloc(sizeof(struct node));
    n->info = x; n->link = first; first = n;
}
void insertEnd(int x) {
    struct node *n = (struct node*) malloc(sizeof(struct node));
    n->info = x; n->link = NULL;
    if (first == NULL) { first = n; return; }
    struct node *t = first;
    while (t->link != NULL) t = t->link;
    t->link = n;
}
void deleteLast() {
    if (first == NULL) { printf("Underflow\n"); return; }
    if (first->link == NULL) { free(first); first = NULL; return; }
    struct node *prev = first, *cur = first->link;
    while (cur->link != NULL) { prev = cur; cur = cur->link; }
    prev->link = NULL; free(cur);
}
void display() {
    for (struct node *t = first; t != NULL; t = t->link) printf("%d -> ", t->info);
    printf("NULL\n");
}
```

Dec-2025 Q2B: create the customer list using dynamic memory allocation

```
struct customer { int id; char name[20]; struct customer *next; };

struct customer* addEnd(struct customer *head, int id, char name[]) {
    struct customer *n = (struct customer*) malloc(sizeof(struct customer));
    n->id = id; strcpy(n->name, name); n->next = NULL;
    if (head == NULL) return n;          // first customer
    struct customer *t = head;
    while (t->next != NULL) t = t->next;  // walk to the last customer
    t->next = n;
    return head;
}
/* head = addEnd(head, 101, "Asha"); head = addEnd(head, 102, "Ravi"); ... */
```

TIP

Adding at the end costs $O(n)$ because we walk the whole list. If you keep an extra pointer **LAST**, insertion at the end becomes $O(1)$. Mention this for extra credit.

COMMON MISTAKES

- Doing **FIRST = NEW** before setting **LINK(NEW)**, which loses the rest of the list.
- Forgetting the empty-list case and the one-node case.
- Forgetting to free the deleted node.
- Moving **FIRST** while walking. Use a separate **TEMP**, **SAVE** or **CUR** pointer.

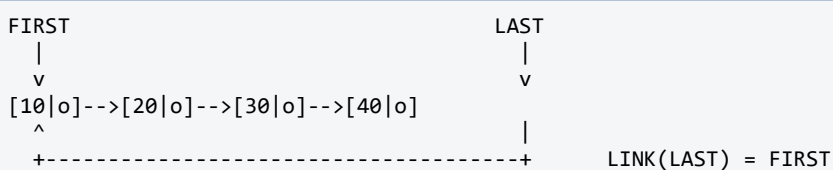
5.4 Circular Linked List ESSENTIAL FOR EXAM

Seen in: Jan-2023 Q3A (10 marks): insertion and deletion at the start, at the end and at any position.

In a **circular linked list** the **LINK** of the last node points back to the **first node** instead of **NULL**. We keep two pointers, **FIRST** and **LAST**.

ANALOGY

Players sitting around a round table passing a ball. After the last player the ball goes back to the first. There is no end of the line.



- **Advantages:** from any node you can reach every other node. With a **LAST** pointer, insertion at the end is $O(1)$. Good for round-robin scheduling and repeating playlists.
- **Disadvantages:** a careless loop can run forever. It is not easy to reverse. Always stop when you return to **FIRST**.

Below are single algorithms that handle **start, end and any position**. **COUNT** is the current number of nodes and **POS** is the position given by the user ($1 = \text{start}$, $\text{COUNT} + 1 = \text{end for insertion}$).

```

Procedure CIR_INSERT(X, POS)
1. If POS < 1 or POS > COUNT + 1 then write "Invalid position"; Return
2. NEW = (node*) malloc(sizeof(node)); INFO(NEW) = X
3. If FIRST = NULL then // list empty
    LINK(NEW) = NEW; FIRST = NEW; LAST = NEW; COUNT = 1; Return
4. If POS = 1 then // insert at START
    LINK(NEW) = FIRST; FIRST = NEW; LINK(LAST) = FIRST
5. Else if POS = COUNT + 1 then // insert at END
    LINK(NEW) = FIRST; LINK(LAST) = NEW; LAST = NEW
6. Else // insert at ANY POSITION
    TEMP = FIRST
    Repeat POS - 2 times: TEMP = LINK(TEMP) // TEMP = node before the position
    LINK(NEW) = LINK(TEMP); LINK(TEMP) = NEW
7. COUNT = COUNT + 1; Return

```

```

Procedure CIR_DELETE(POS)
1. If FIRST = NULL then write "Underflow: list is empty"; Return
2. If POS < 1 or POS > COUNT then write "Invalid position"; Return
3. If COUNT = 1 then // only one node
    free(FIRST); FIRST = NULL; LAST = NULL; COUNT = 0; Return
4. If POS = 1 then // delete from START
    OLD = FIRST; FIRST = LINK(FIRST); LINK(LAST) = FIRST
5. Else // delete from END or ANY POSITION
    PREV = FIRST
    Repeat POS - 2 times: PREV = LINK(PREV) // PREV = node before the target
    OLD = LINK(PREV)
    LINK(PREV) = LINK(OLD)
    If OLD = LAST then LAST = PREV // deleting the last node
6. free(OLD); COUNT = COUNT - 1; Return

```

Example: list 10 -> 20 -> 30 (circular), COUNT = 3
 CIR_INSERT(5, 1) : 5 -> 10 -> 20 -> 30, LINK(LAST) = FIRST (the 5 node)
 CIR_INSERT(40, 5) : end, 5 -> 10 -> 20 -> 30 -> 40, LAST = the 40 node
 CIR_INSERT(15, 3) : TEMP = 10 (walk 1 step), result 5 -> 10 -> 15 -> 20 -> 30 -> 40
 CIR_DELETE(6) : deletes 40, PREV = 30 becomes LAST and LINK(30) = FIRST

📌 EXAM TIP

For a 10-mark question, also draw the circular diagram and one before/after picture for the start, the end and the middle case. If time is short, write the three insertion cases and three deletion cases as separate small algorithms.

5.5 Doubly Linked List ★ ESSENTIAL FOR EXAM

Seen in: Jan-2022 Q1C (OR option, 6 marks): add an element at the end of a doubly linked list.

Each node has **two pointers**: **LPTR** (previous node) and **RPTR** (next node), plus **INFO**. The pointer **L** marks the leftmost node and **R** the rightmost node.

🧠 ANALOGY

A metro line where you can travel forwards and backwards. A singly list is a one-way street.

```

NULL <-- [LPTR|10|RPTR] <--> [LPTR|20|RPTR] <--> [LPTR|30|RPTR] --> NULL
          ^                               ^
          L                               R

Node:  +-----+-----+-----+
        | LPTR | INFO | RPTR |
        +-----+-----+-----+

```

- **Advantages:** traverse in both directions. A node can be deleted when only its own address is known.
- **Disadvantage:** one extra pointer per node, and insertion and deletion update more pointers.

```

Procedure DOU_INS_END(X, L, R)                                // insert at the end
1. NEW = (node*) malloc(sizeof(node)); INFO(NEW) = X; RPTR(NEW) = NULL
2. If R = NULL then                                          // list empty
    LPTR(NEW) = NULL; L = NEW; R = NEW; Return
3. LPTR(NEW) = R                                              // new node points back to the old last node
4. RPTR(R) = NEW                                              // old last node points forward to new node
5. R = NEW                                                    // R now marks the new last node
6. Return

```

BEFORE: L --> [10] <--> [20] <-- R
AFTER : L --> [10] <--> [20] <--> [30] <-- R (insert 30)

```

Procedure DOU_INS_FRONT(X, L, R)
1. NEW = malloc node; INFO(NEW) = X
2. LPTR(NEW) = NULL
3. If L = NULL then
    RPTR(NEW) = NULL; L = NEW; R = NEW
4. Else
    RPTR(NEW) = L; LPTR(L) = NEW;
    L = NEW
5. Return

```

```

Procedure DOU_DEL(L, R, OLD)
1. If R = NULL then Underflow; Return
2. If L = R then L = NULL; R = NULL (one node)
3. Else if OLD = L then
    L = RPTR(L); LPTR(L) = NULL
4. Else if OLD = R then
    R = LPTR(R); RPTR(R) = NULL
5. Else (a middle node)
    RPTR(LPTR(OLD)) = RPTR(OLD)
    LPTR(RPTR(OLD)) = LPTR(OLD)
6. free(OLD)

```

Quick revision box: Unit 5

Topic	Remember
Singly list node	INFO + LINK. FIRST points to the first node. Last LINK = NULL.
Insert at front	LINK(NEW) = FIRST, then FIRST = NEW (this order!).
Insert at end	Walk until LINK(SAVE) = NULL, then LINK(SAVE) = NEW. Handle the empty list.
Delete last	Walk with PREV and CUR. LINK(PREV) = NULL, then free(CUR). Handle one node and empty list.
Circular list	Last LINK points to FIRST. Keep FIRST and LAST. Update LINK(LAST) when FIRST changes.
Doubly list	LPTR + INFO + RPTR. When inserting, set the new node first, then change the neighbours.
Dynamic allocation	malloc on insert, free on delete. Flexible, no wasted space.

Complexity Summary, Practice Paper and Final Cheat Sheet

6.1 Time Complexity of Each Structure (end of your syllabus slides)

★ ESSENTIAL FOR EXAM

Structure	Access	Search	Insert	Delete
Array	O(1)	O(n)	O(n) (shifting)	O(n) (shifting)
Stack	O(n)	O(n)	O(1) (PUSH)	O(1) (POP)
Queue	O(n)	O(n)	O(1) (ENQUEUE)	O(1) (DEQUEUE)
Singly linked list	O(n)	O(n)	O(1) at front, O(n) at end*	O(1) at front, O(n) at end

*O(1) at the end if you keep a LAST pointer. In a doubly linked list, deleting a node whose address you already have is O(1).

6.2 Self-Test: Practice Paper (mirrors the past-paper pattern)

Time yourself: about 45 minutes. Attempt without looking, then check the answer key below.

Q	Question	Marks
1A	Differentiate between linear and non-linear data structures with two applications.	4
1B	An array A[1..8][1..6] has Base = 300 and each element takes 4 bytes. Find the address of A[5][2] in row-major and column-major order.	6
1C	Represent $7X^2Y + 5XY^2 + 3X + 9$ using a singly linked list. Draw the nodes, give the node format and write an algorithm to count the terms.	6
2A	Convert $A + B * (C - D) / E$ to postfix. Then evaluate it for A = 6, B = 4, C = 9, D = 5, E = 2 using a stack.	6
2B	Justify: "A circular queue is better than a simple queue under static memory." Write the algorithm to insert into a simple queue.	6
2C	Insert a node at the front and delete a node from the end of a singly linked list (algorithms).	6
3A	Show F and R after each step for a circular queue of size 5: insert(10), insert(20), insert(30), delete(), insert(40), insert(50), insert(60), insert(70).	8
3B	Write an algorithm to insert a node at the end of a doubly linked list. Show a diagram.	6

Answer key

Q	Answer
1A	Use the table in 1.3 (arrangement, levels, traversal, memory, examples). Applications: stack for undo, queue for printing; tree for folders, graph for maps.
1B	R = 8, C = 6. Row-major: $300 + 4 \times [(5-1) \times 6 + (2-1)] = 300 + 4 \times 25 = 400$. Column-major: $300 + 4 \times [(2-1) \times 8 + (5-1)] = 300 + 4 \times 12 = 348$.
1C	Terms: (7,2,1), (5,1,2), (3,1,0), (9,0,0): 4 nodes with fields COEFF, PX, PY, LINK. COUNT_TERMS returns 4.
2A	Postfix: $**A B C D - * E / + **$. With the values: $9-5 = 4$; $4*4 = 16$; $16/2 = 8$; $6+8 = 14$.
2B	Use the model answer in 4.2 plus ENQUEUE from 4.1.
2C	Use algorithms A and C in 5.3.
3A	F = 2, R = 1 after insert(60), and the queue holds 20, 30, 40, 50, 60 (full). insert(70) gives OVERFLOW . Trace: (F,R) = (1,1), (1,2), (1,3), delete (2,3), (2,4), (2,5), then R wraps (2,1).
3B	Use DOU_INS_END in 5.5.

6.3 One-Page Final Cheat Sheet

Topic	What to write or remember
Row-major address	$\text{Base} + w \times [(i - \text{LBr}) \times C + (j - \text{LBC})]$

Topic	What to write or remember
Column-major address	$\text{Base} + w \times [(j - \text{LBC}) \times R + (i - \text{LBr})]$
R and C	Upper - Lower + 1 for each dimension
Infix to postfix	Operand out. (push.) pop to (. Operator: pop while top $\geq$ it, then push. End: pop all.
Postfix evaluation	Operand push. Operator: pop B, pop A, push A op B.
Stack	LIFO, TOP, Overflow / Underflow, empty when TOP = 0.
Simple queue	FIFO, F and R, empty when F = 0, R never moves back (false overflow).
Circular queue	Wrap N to 1. Full when F = R + 1 or (F = 1 and R = N). Reuses space.
Priority queue	One row per priority. Delete from the first non-empty row.
Singly list node	INFO + LINK; FIRST; last LINK = NULL.
Polynomial node	COEFF + powers of each variable + LINK. One node per term.
Circular list	LINK(LAST) = FIRST. Keep FIRST and LAST.
Doubly list	LPTR + INFO + RPTR. L and R pointers.
Static vs dynamic	Array, fixed size, compile time vs linked list, malloc and free, run time.
Linear search	Best $O(1)$, average $O(n)$, worst $O(n)$. Binary search is $O(\log n)$ on sorted data.
Linear vs non-linear	Sequence (array, stack, queue, list) vs hierarchy or network (tree, graph).

📌 EXAM TIP

Exam-day checklist

- Read every question, mark the 6-mark numericals and do them first. They are the safest marks.
- Always write the formula or rule before substituting.
- Draw the diagram first for linked-list and queue questions.
- Show every step in tables for postfix, queue and stack traces.
- For “justify” questions: statement $\rightarrow$ reason $\rightarrow$ example $\rightarrow$ conclusion.
- For “write an algorithm” questions: check empty or full, do the main steps, update pointers, return.